INFORMATICS INSTITUTE OF TECHNOLOGY

UNIVERSITY OF WESTMINSTER

MODULE: (2026) 6COSC022C.2 Advanced Server-Side Web Programming

COURSE-WORK REPORT

Name: G.P.S.Costa

IIT ID: 20211404

UOW ID: W1957449

Tutorial Group : 07

Table of Contents

List of Figures iii

01.Introduction 1

1.1 Overview of the System 1

1.2 Problem Statement 1

1.3 Proposed Solution 1

2. System Overview 2

2.1 System Objectives 2

2.2 System Feature Overview 2

3. System Architecture 3

3.1 Overall High-Level Backend Architecture 3

3.3 MVC Architecture 5

3.4 Security Architecture 5

3.5 Routing Architecture 6

4. Flow Diagrams 7

4.1 Bidding System Flow Diagram 7

5. Database Design 9

5.1 Entity Relationship Diagram 9

5.2 Enhanced Entity Relationship Diagram 10

6. Security Implementation 11

7. Nodemailer Integration 11

8. Background Task Scheduling for Bid Cycle Finalization 11

9. API Design and End Points 12

9.1 End Points Summary Table 12

10.2 API Testing Screenshots 14

10.2.1 Register Scenario 14

10.2.2 Account Verification 14
10.2.3 Login 14
10.2.4 Forgot Password 15
10.2.5 Creating User Profile 15
10.2.6 Bidding Scenario 15
11. MySQL Database 17
12. UI Screenshots 17
13. System Workflow 19
14. Conclusion 19
15. Appendix 20

List of Figures

Figure 1: Overall High - Level Backend Architecture 3
Figure 2: Overall High- Level Server Side Architecture 4
Figure 3: MVC Architecture 5
Figure 4: Security Architecture 5
Figure 5: Routing Architecture 6
Figure 6: Bidding System Flow Diagram 7
Figure 7: Server Side Workflow Diagram 8
Figure 8: Filter Process Workflow Diagram 8
Figure 9: Report Generation Workflow Diagram 8
Figure 10: Entity Relationship Diagram 9
Figure 11: Enhanced Entity Relationship Diagram 10
Figure 12: User Registration Verification 14
Figure 13: User Registration 14
Figure 14: Invalid Domain Registration 14
Figure 15: Weak Password Registration 14
Figure 16: User Account Verification 14
Figure 17: User Login 14
Figure 18: Forgot Password 15
Figure 19: Forgot Password Reset Email 15
Figure 20: Creating User Profile 15
Figure 21: User A - Initial Bid Notification Email 15
Figure 22: User A - Initial Bid 15
Figure 23: User B - Bid Limit Excedd 16
Figure 24: User B Wins 16
Figure 25: User B - Wins Notification Email 16
Figure 26: Monthly Exceed Limit 16
Figure 27: MySQL Database 17
Figure 28: Register Page UI 17
Figure 29: Login Page UI 17
Figure 30: Bidding Page UI 18
Figure 31: Data Analytics Dashboard UI 18
Figure 32: Descriptive System Workflow Diagram 20
Figure 33: Descriptive Dashboard Workflow Diagram 21

01.Introduction

1.1 Overview of the System

The Alumni Influencer Platform is a web-based API system developed to enhance alumni engagement through a modern and interactive approach. The platform enables alumni to showcase their professional achievements, connect with students, and participate in a dynamic ecosystem driven by sponsorship and visibility.

The system is designed as a backend API that can be integrated with multiple client applications, including web and augmented reality (AR) interfaces. It provides structured data access, secure authentication, and scalable architecture to support real-time interactions between users and the platform.

1.2 Problem Statement

The existing alumni system is based on a static webpage that presents limited and often outdated information about graduates. This approach results in low student engagement, minimal interaction between alumni and current students, and a lack of visibility into alumni career progression.

Additionally, the current system does not support dynamic content updates, professional endorsements, or interactive features such as bidding or sponsorship. As a result, it fails to motivate alumni participation and does not provide meaningful value to students seeking career inspiration.

1.3 Proposed Solution

To address these limitations, this project introduces a dynamic API-based system that enables alumni to create detailed profiles and participate in a blind bidding process to be featured as the “Alumni of the Day.” The system enforces rules such as monthly limits and event-based eligibility to ensure fairness and engagement.

The application is developed using Node.js and Express.js with Sequelize ORM and a MySQL database. Security is ensured through JWT-based authentication and encrypted password storage, while the API remains client-agnostic for integration with various applications.

2. System Overview

2.1 System Objectives

The system is a backend API-driven platform designed to manage alumni engagement through a structured and scalable architecture. It operates without a dedicated frontend, instead exposing RESTful API endpoints that can be accessed by various clients such as web applications, mobile apps, or AR-based interfaces. Alumni users interact with the system by registering, creating professional profiles, and participating in the bidding process, while developers consume the API to retrieve and display data such as the "Alumni of the Day." The system processes requests through organized routes and controllers, applies business logic such as bidding rules and eligibility checks, and securely stores and retrieves data from the database.

2.2 System Feature Overview

Table 1: System Feature Overview

Feature	Description
Authentication System	Enables secure user registration using university email addresses, login with credential validation, email verification to ensure authenticity, and password reset functionality for account recovery.
Profile Management	Allows alumni to create and manage detailed professional profiles including personal information, LinkedIn URLs, degrees, certifications, licences, short courses, and employment history.
Bidding System	Implements a blind bidding mechanism where alumni compete to become the "Alumni of the Day." The system enforces rules such as increase-only bids, monthly win limits, and an additional bid opportunity through alumni event participation.
API Security	Secures endpoints using JWT-based authentication and bearer tokens. Middleware validates tokens and restricts access to authorized users while ensuring safe data handling.
Public API	Provides open access endpoints for retrieving featured alumni data, allowing external clients such as AR applications to display the "Alumni of the Day."

3. System Architecture

3.1 Overall High-Level Backend Architecture

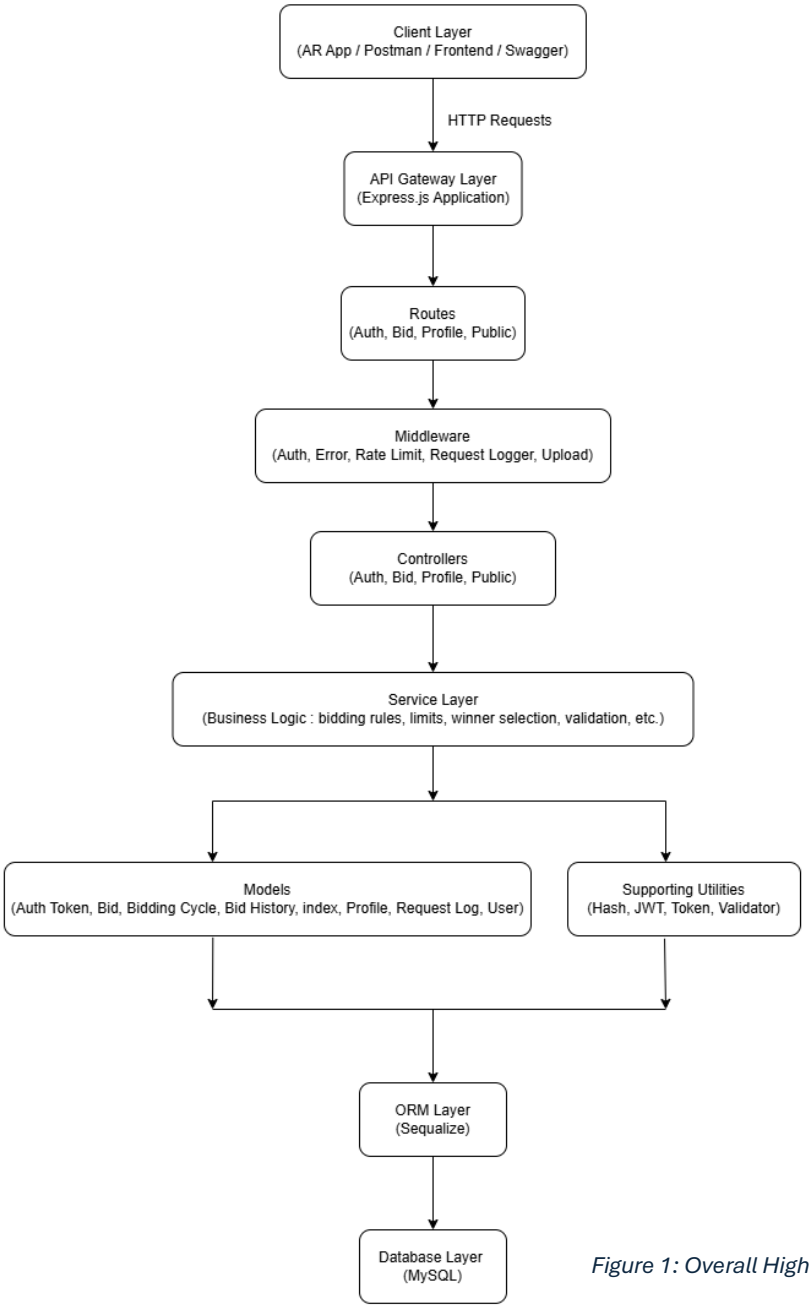

Figure 1: Overall High - Level Backend Architecture

The overall architecture follows a layered API design. Client applications communicate with an Express-based backend, which coordinates authentication, business services, and persistence through Sequelize ORM. Supporting services such as email delivery and public API exposure are connected as separate concerns around the main core application. This separation improves clarity, maintainability, and extensibility.

3.2 Overall High-Level Server-Side Architecture

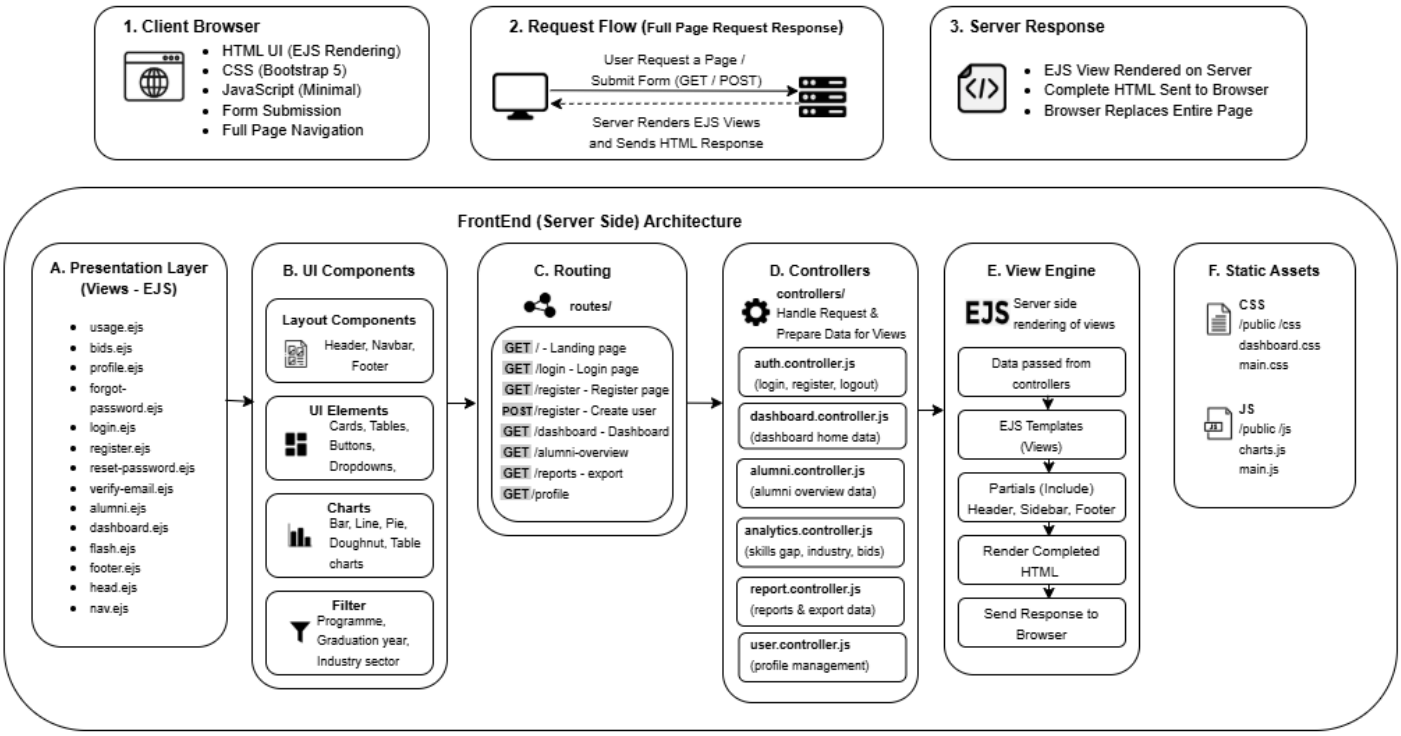

Figure 2: Overall High- Level Server Side Architecture

The frontend architecture follows a server-side layered presentation design. Client browsers communicate with the Express application through page requests and form submissions, where routes and controllers process the request, prepare the required data, and render EJS templates on the server. The interface is organised into EJS views, reusable layout components, UI elements, charts, filters, and static assets such as CSS and JavaScript files. Once the view engine renders the completed HTML, the browser receives and displays the full page response. This structure separates presentation, routing, controller logic, and static resources, improving clarity, maintainability, and extensibility for future interface and dashboard enhancements.

3.3 MVC Architecture

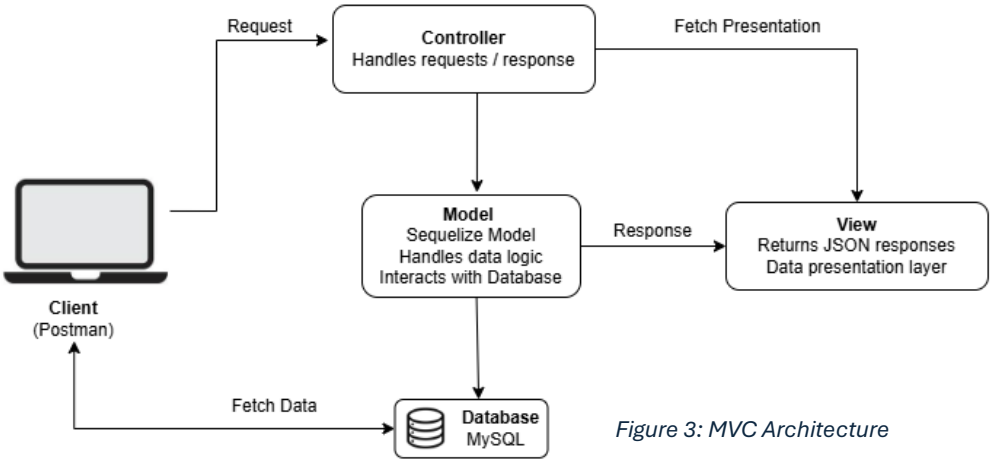

Figure 3: MVC Architecture

Although the system does not use server-rendered views in the traditional web sense, it still follows an MVC-inspired backend pattern. Routes and controllers receive requests, services implement business logic, and models define the database entities. JSON responses effectively replace the view layer, making this a practical API adaptation of MVC.

3.4 Security Architecture

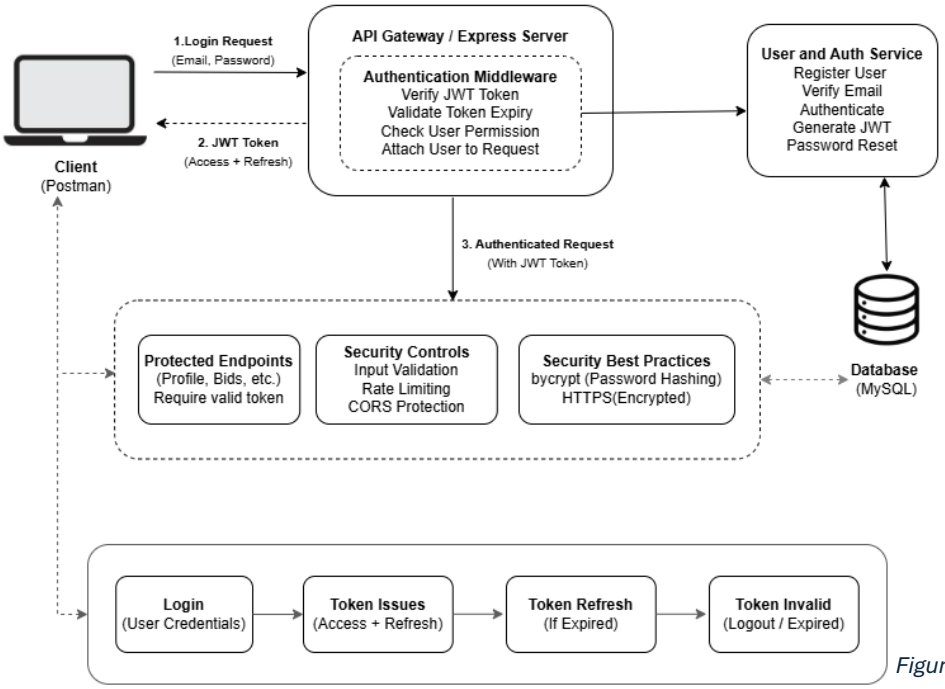

Figure 4: Security Architecture

Security is implemented as a layered concern rather than a single feature. Input validation, bearer-token verification, token expiry checks, password hashing, and API hardening measures work together to protect the platform. This reduces the chance of unauthorized access, insecure storage, and weak request handling.

3.5 Routing Architecture

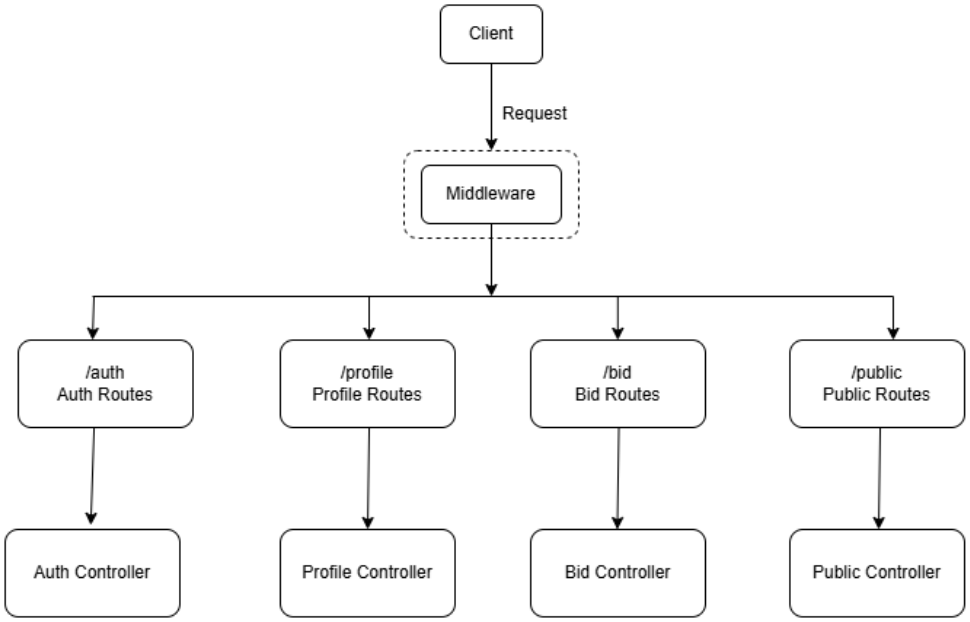

Figure 5: Routing Architecture

The routing structure groups related endpoints into authentication, profile, bid, and public modules. Middleware is applied selectively so that protected operations require valid tokens while public routes remain accessible. This organization improves readability in the codebase and simplifies endpoint maintenance.

4. Flow Diagrams

4.1 Bidding System Flow Diagram

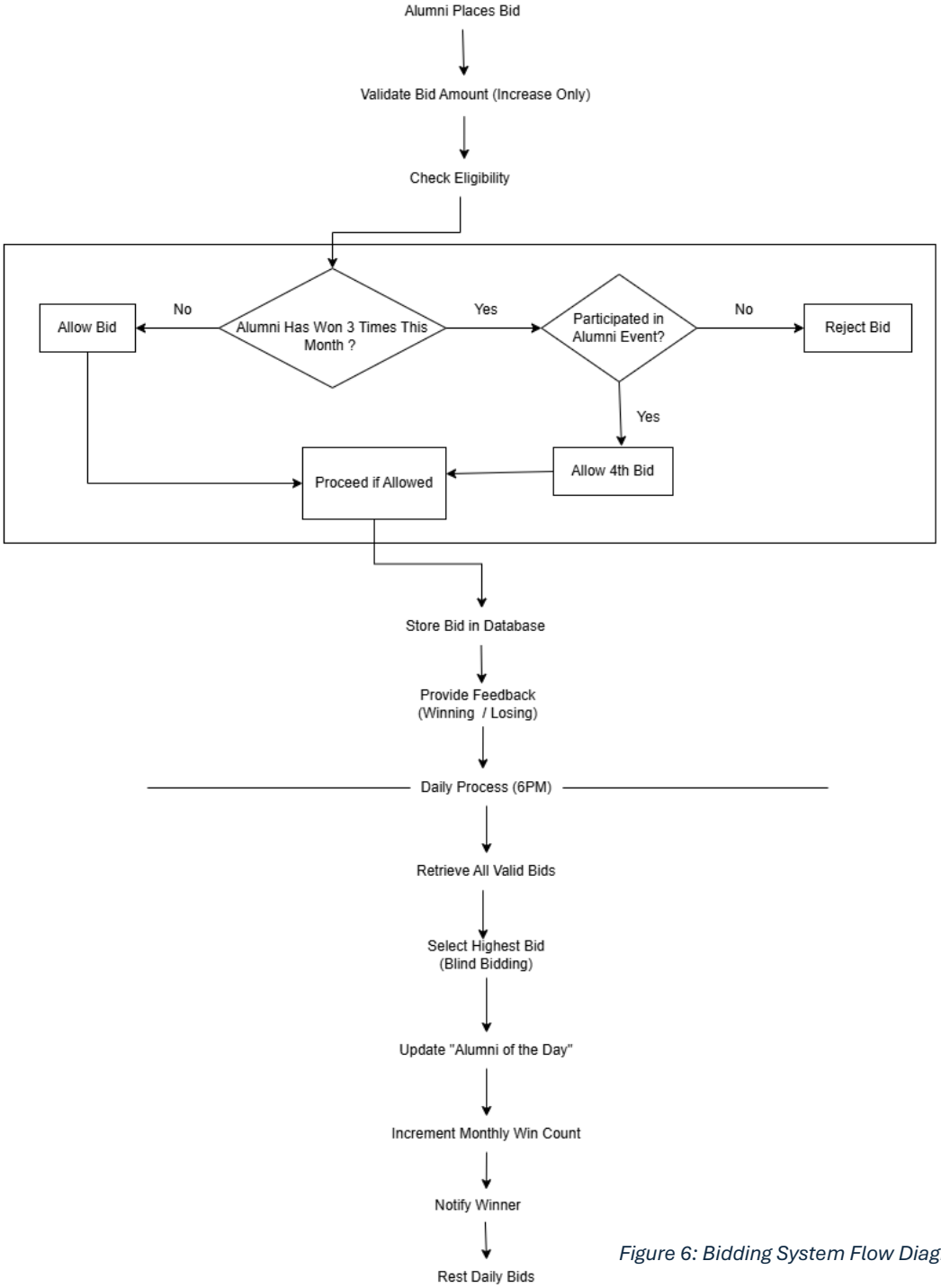

Figure 6: Bidding System Flow Diagram

The bidding flow validates the request, checks standard participation limits, evaluates whether an event-based extra bid is available, stores the blind bid, and later processes winner selection when the relevant cycle closes. This preserves business consistency while keeping the user-facing interaction simple.

4.2 Server - Side Workflow Diagram

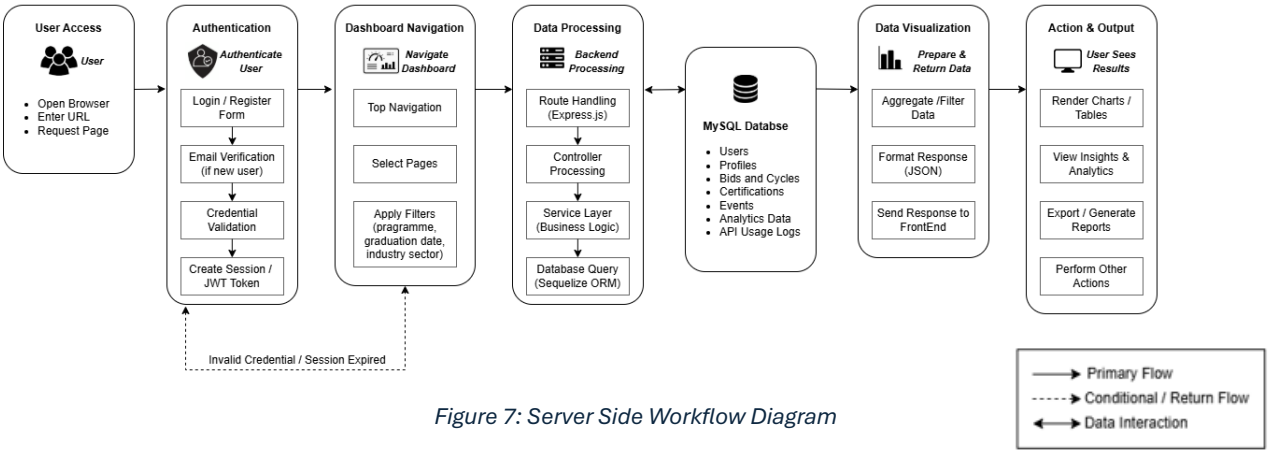

Figure 7: Server Side Workflow Diagram

4.3 Filter Process Workflow

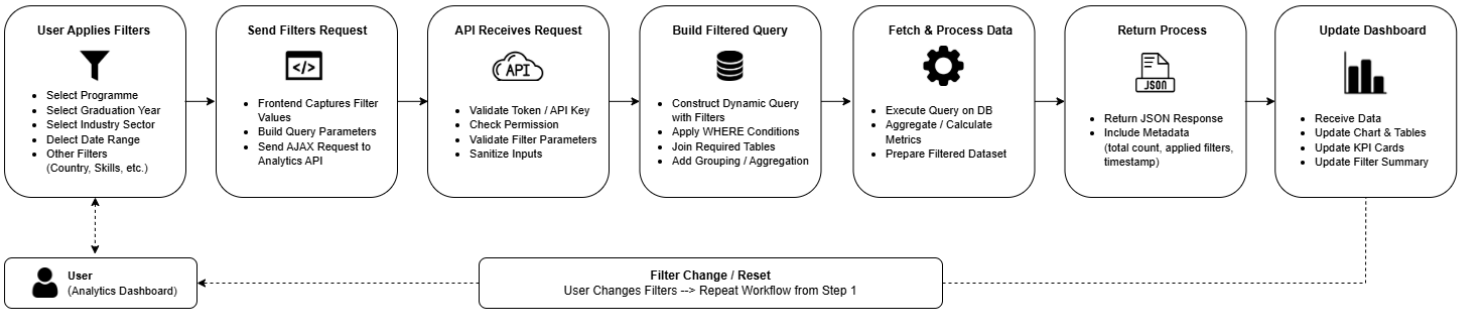

Figure 8: Filter Process Workflow Diagram

4.4 Report Generation Workflow

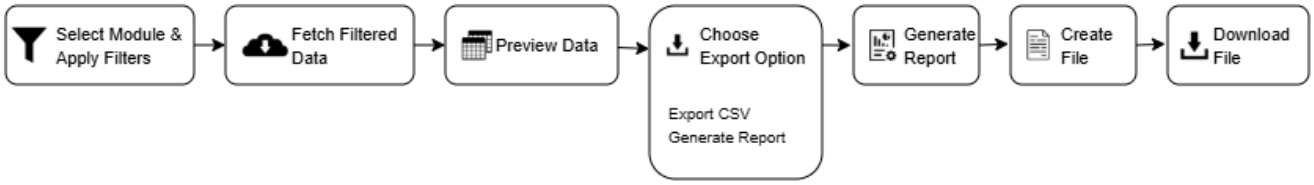

Figure 9: Report Generation Workflow Diagram

5. Database Design

5.1 Entity Relationship Diagram

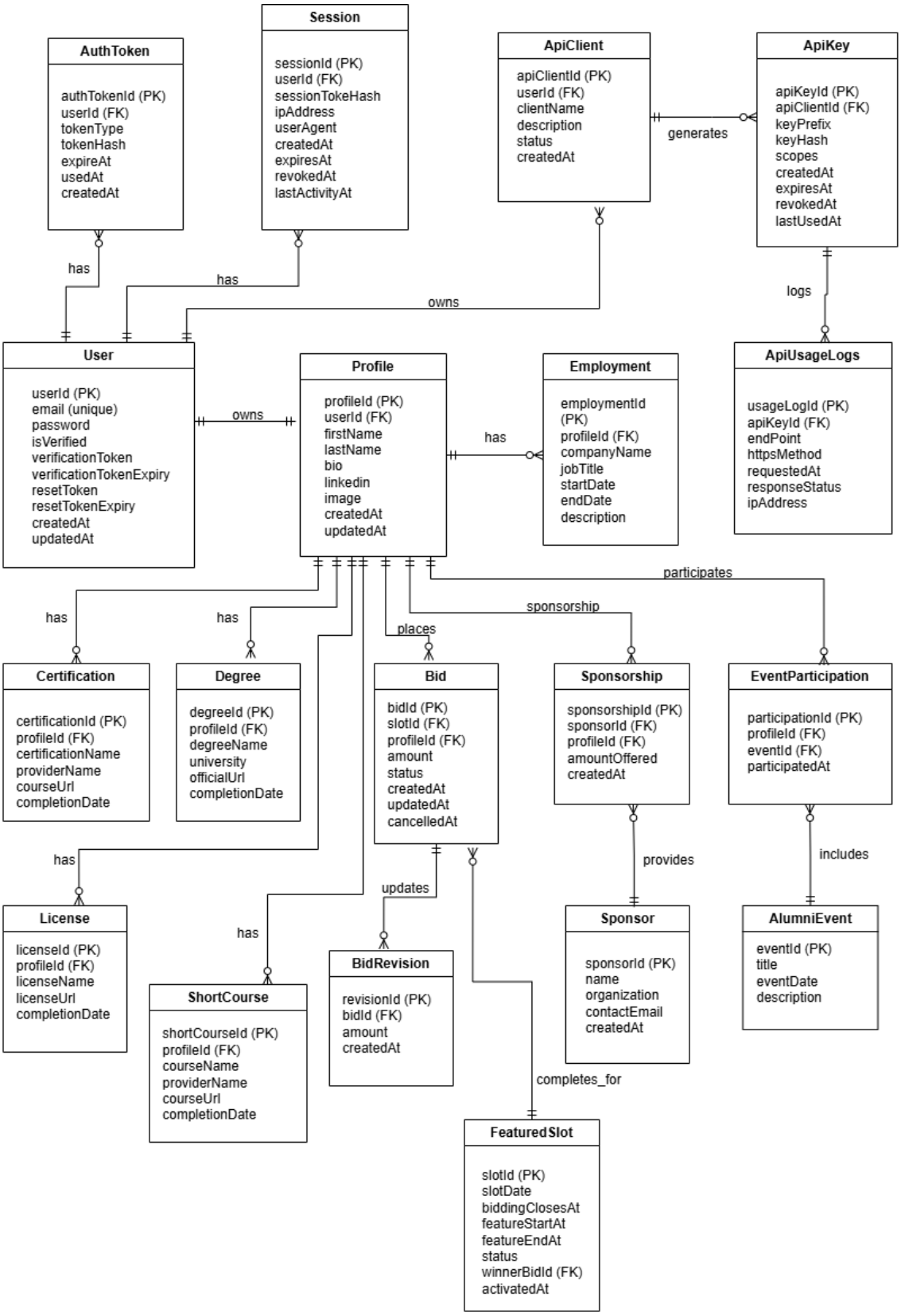

Figure 10: Entity Relationship Diagram

5.2 Enhanced Entity Relationship Diagram

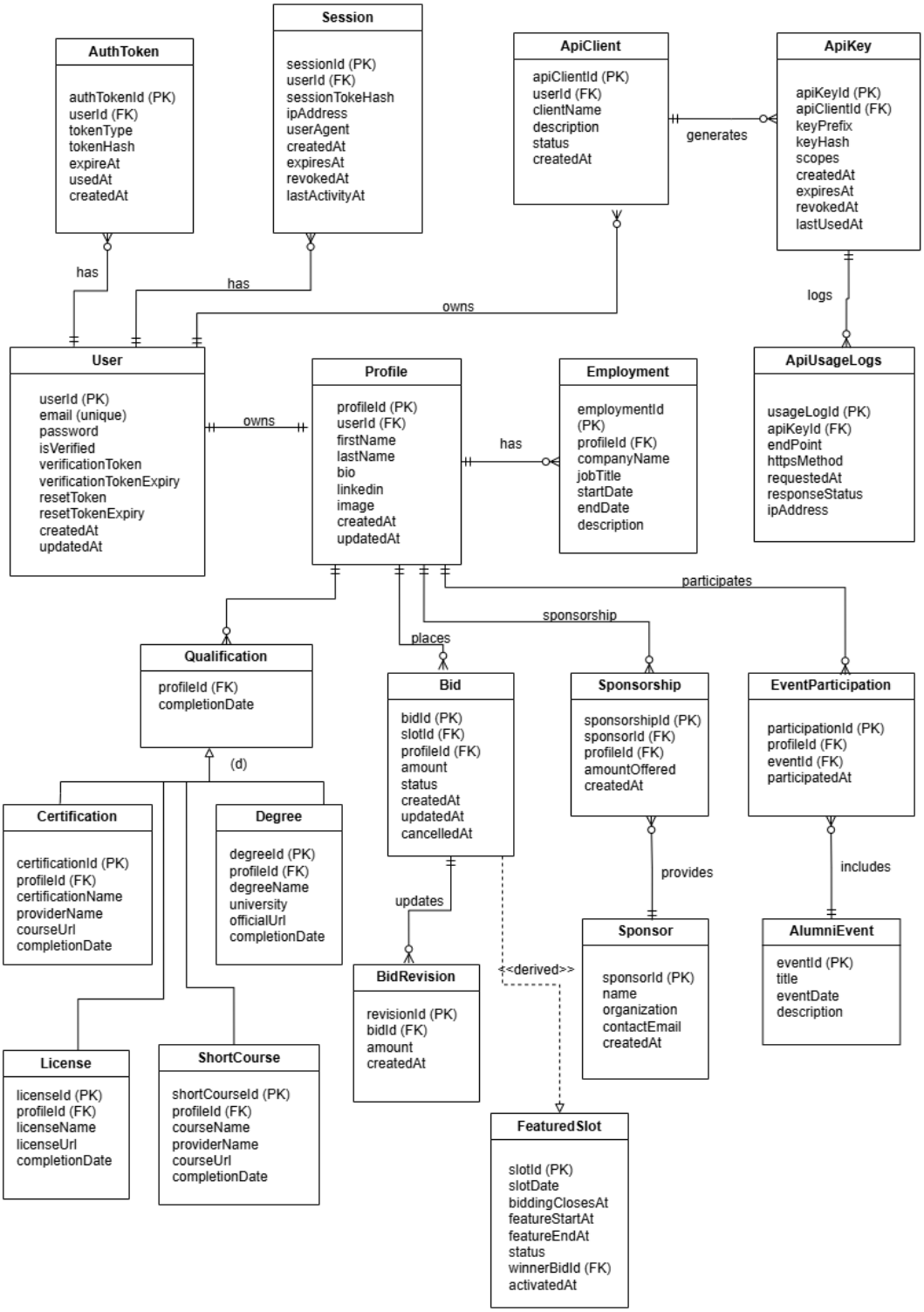

Figure 11: Enhanced Entity Relationship Diagram

6. Security Implementation

Password hashing: Passwords are stored using bcrypt hashing rather than plaintext storage. This ensures that even if database access is compromised, the original password values are not directly exposed.

JWT authentication: After successful login, the user receives a signed bearer token that is sent with later requests. Protected routes verify this token before allowing access.

Input validation: Request bodies and parameters are validated before processing. This reduces invalid data entry and lowers the risk of common backend issues caused by malformed input.

Token expiry: Verification, reset, and session tokens are issued with expiry control so that old tokens cannot remain valid indefinitely.

7. Nodemailer Integration

Nodemailer was used in the alumni engagement platform to manage email-based communication features such as account verification, password reset support, and bidding-related notifications. It provided a simple and flexible way to connect the backend with an email delivery service, allowing the system to automatically send important messages to users when specific actions or events occurred.

To support both local testing and production use, the email setup was designed to work in two modes. During local development and testing, emails could be captured using a local mail testing service, which made it possible to verify email content, subject lines, tokens, and notification flows without sending real messages to users. In the production-ready environment, the same Nodemailer-based implementation could be connected to a real SMTP provider so that actual emails could be delivered securely to alumni users. This approach improved testing reliability, reduced the risk of sending unintended emails during development, and ensured that the platform's communication features remained consistent across both testing and deployment environments.

8. Background Task Scheduling for Bid Cycle Finalization

A background task was implemented using a cron-based scheduler to ensure that bidding cycles are finalized automatically at the correct time. This allows the system to regularly check active cycles, process cycle completion logic, and move the bidding workflow forward without

requiring manual administrator action. Using cron for this purpose made the process simple, reliable, and suitable for both local testing and production deployment, especially for managing time-sensitive bid cycle operations.

9. API Design and End Points

9.1 End Points Summary Table

Table 2: End Points Summary

Route Name	Method	Route Path	Description
Register	POST	/auth/register	Registers a new alumnus account using email and password.
Verify Email	GET	/auth/verify-email	Verifies a user's email address using a verification token.
Login	POST	/auth/login	Authenticates a user and returns a JWT access token.
Verify Token	GET	/auth/verify-token	Checks whether the current JWT token is valid.
Forgot Password	POST	/auth/forgot-password	Creates a password reset token for a user who forgot their password.
Reset Password	POST	/auth/reset-password	Resets the user's password using a valid reset token.
Usage Statistics	GET	/auth/usage	Returns JWT usage statistics for the authenticated alumnus.
Logout	POST	/auth/logout	Logs out the authenticated user by invalidating the current JWT version.
Current Cycle	GET	/bids/current-cycle	Retrieves the currently active bidding cycle.
Place Bid	POST	/bids	Allows an authenticated alumnus to place a blind bid in the active cycle.

Update Bid	PATCH	/bids/:bidId	Increases or updates an existing bid in the current cycle.
Cancel Bid	DELETE	/bids/:bidId	Cancels an active bid in the current cycle.
My Bids	GET	/bids/me	Lists all bids placed by the authenticated alumnus across cycles.
Bid Status	GET	/bids/status	Shows whether the user is currently winning or losing in the active cycle.
Cycle History	GET	/bids/cycles/history	Retrieves the history of active and processed bidding cycles with related bid records.
Cycle History by ID	GET	/bids/cycles/:cycleId	Retrieves one specific bidding cycle and its history details.
Process Current Cycle	POST	/bids/process-current-cycle	Manually processes the current cycle, mainly for local testing.
Get My Profile	GET	/profile/me	Retrieves the authenticated alumnus profile details.
Create or Replace Profile	POST	/profile	Creates a new profile or replaces the existing profile for the authenticated alumnus.
Update Profile	PUT	/profile	Updates the authenticated alumnus profile information.
Delete Profile	DELETE	/profile	Deletes the authenticated alumnus profile.
Today's Featured Alumnus	GET	/public/featured/today	Retrieves the alumnus featured for today on the public endpoint.

10.2 API Testing Screenshots

10.2.1 Register Scenario

10.2.1.1 Success Scenario

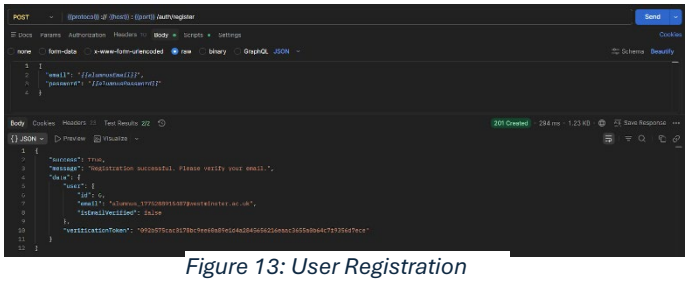

Figure 13: User Registration

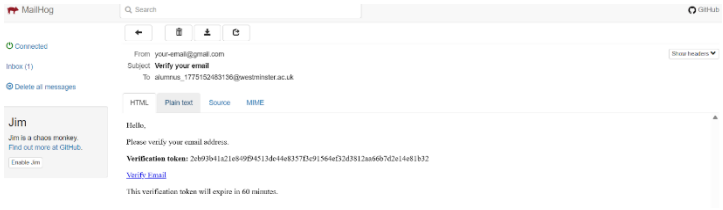

Figure 12: User Registration Verification

10.2.1.2 Negative Scenarios

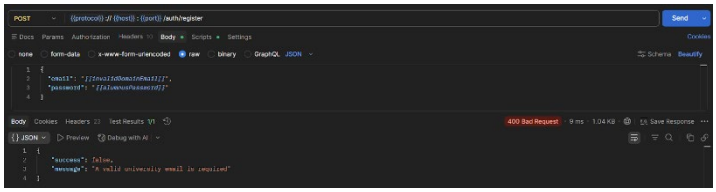

Figure 14: Invalid Domain Registration

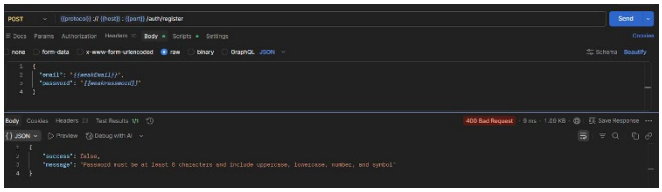

Figure 15: Weak Password Registration

10.2.2 Account Verification

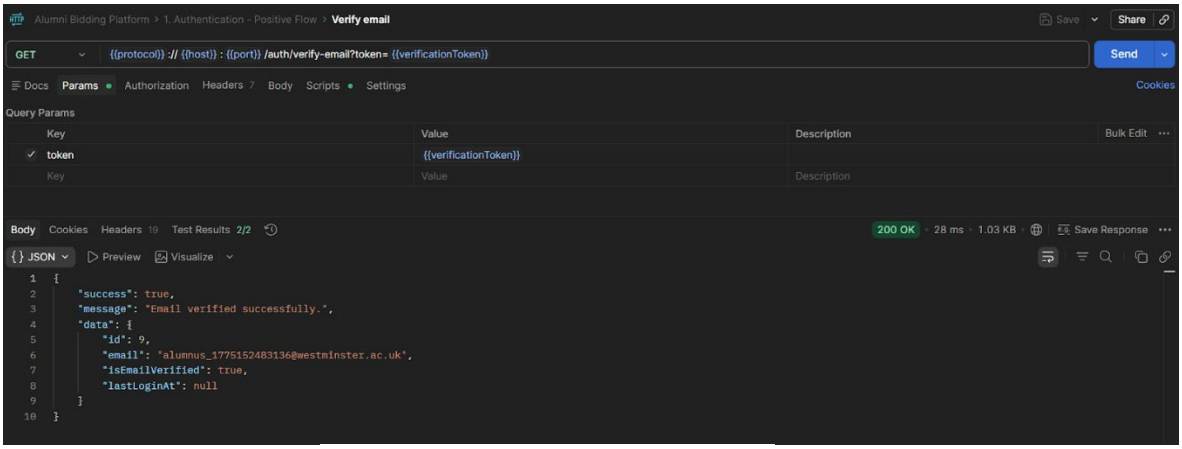

Figure 16: User Account Verification

10.2.3 Login

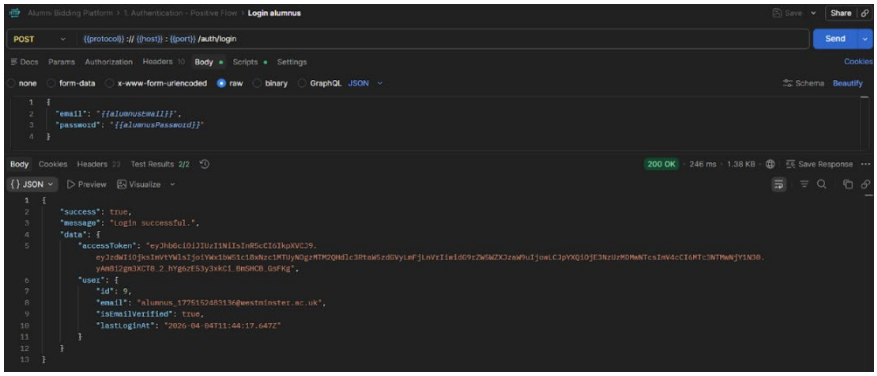

Figure 17: User Login

10.2.4 Forgot Password

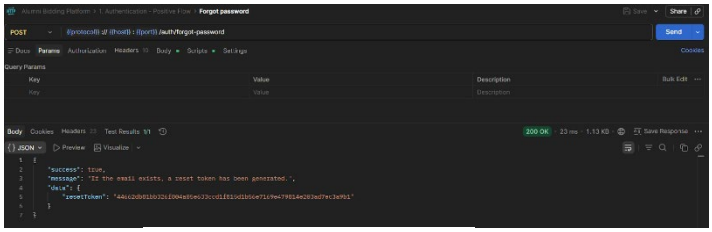

Figure 18: Forgot Password

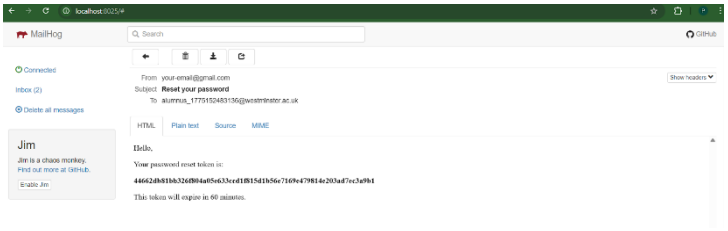

Figure 19: Forgot Password Reset Email

10.2.5 Creating User Profile

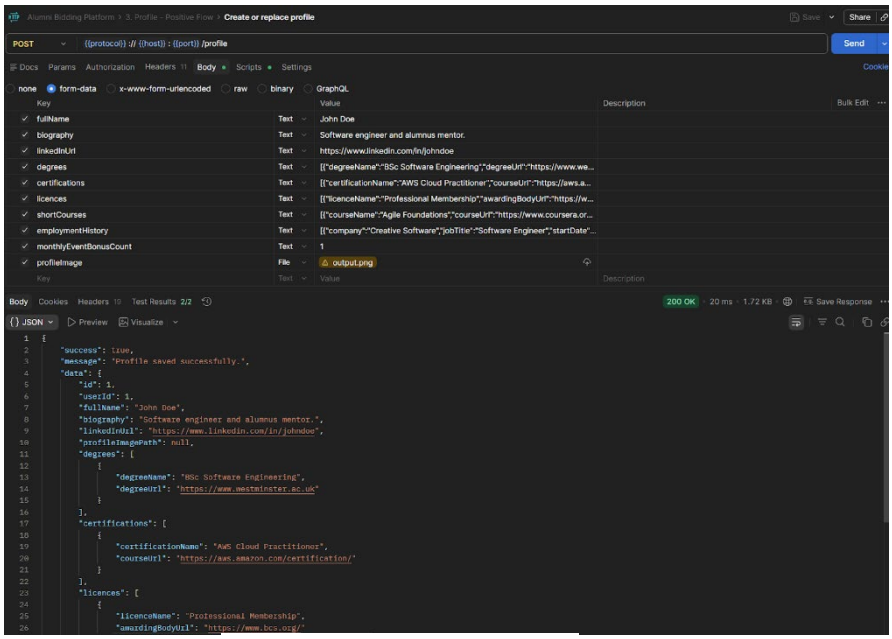

Figure 20: Creating User Profile

10.2.6 Bidding Scenario

10.2.6.1 User A - Place Initial Bid

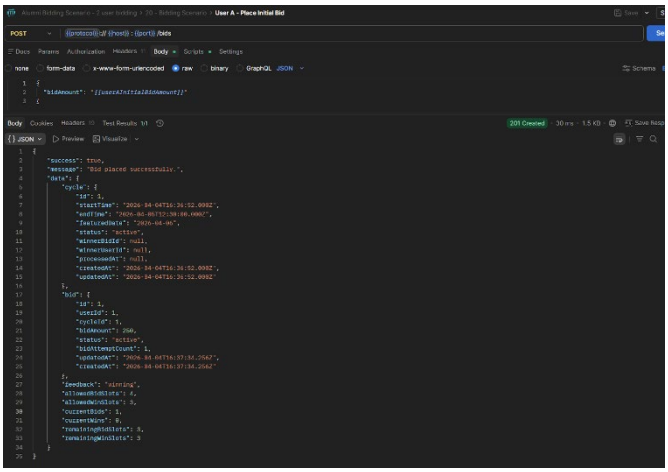

Figure 22: User A - Initial Bid

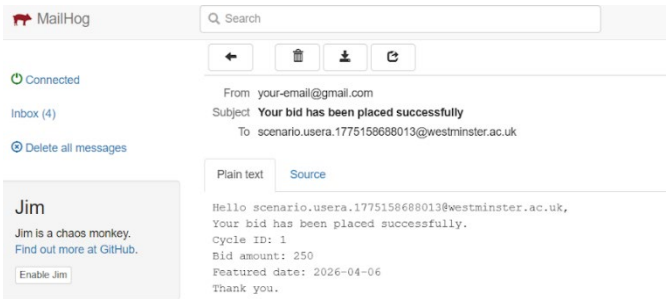

Figure 21: User A - Initial Bid Notification Email

10.2.6.2 User B – Use 3 Slots and try for 4th Bid

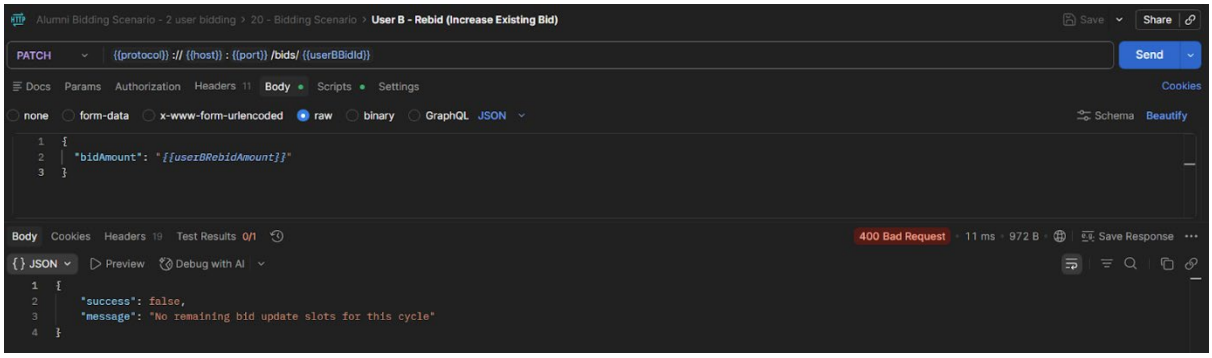

Figure 23: User B - Bid Limit Excedd

10.2.6.3 User B – Wins

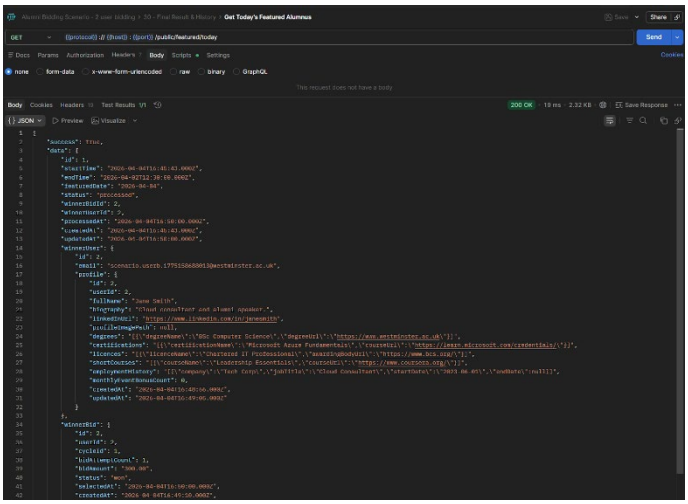

Figure 24: User B Wins

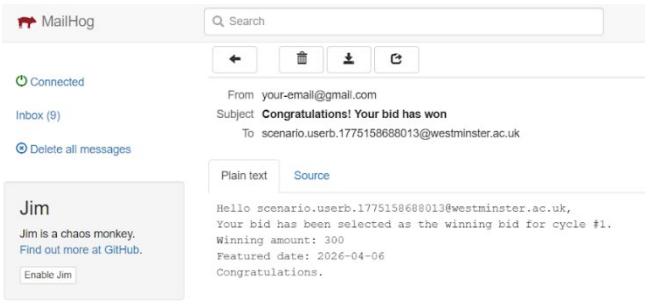

Figure 25: User B - Wins Notification Email

10.2.6.4 User B - Won 3 "Alumni of the Day" 3 times and try to Bid for 4th Time

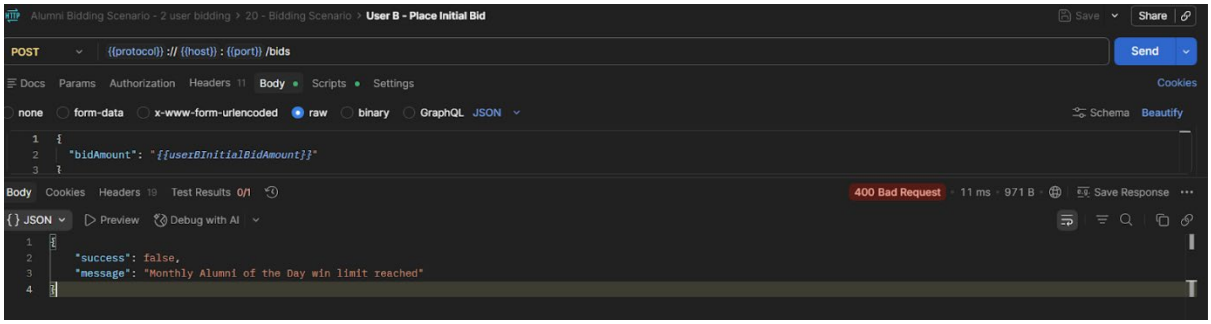

Figure 26: Monthly Exceed Limit

11. MySQL Database

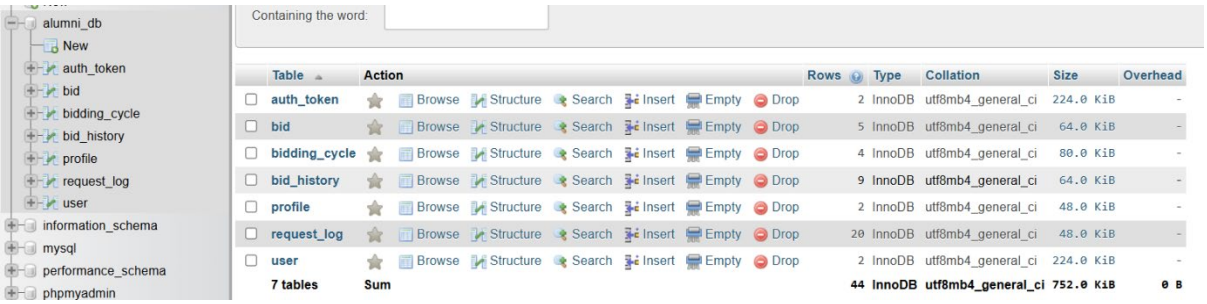

Figure 27: MySQL Database

12. UI Screenshots

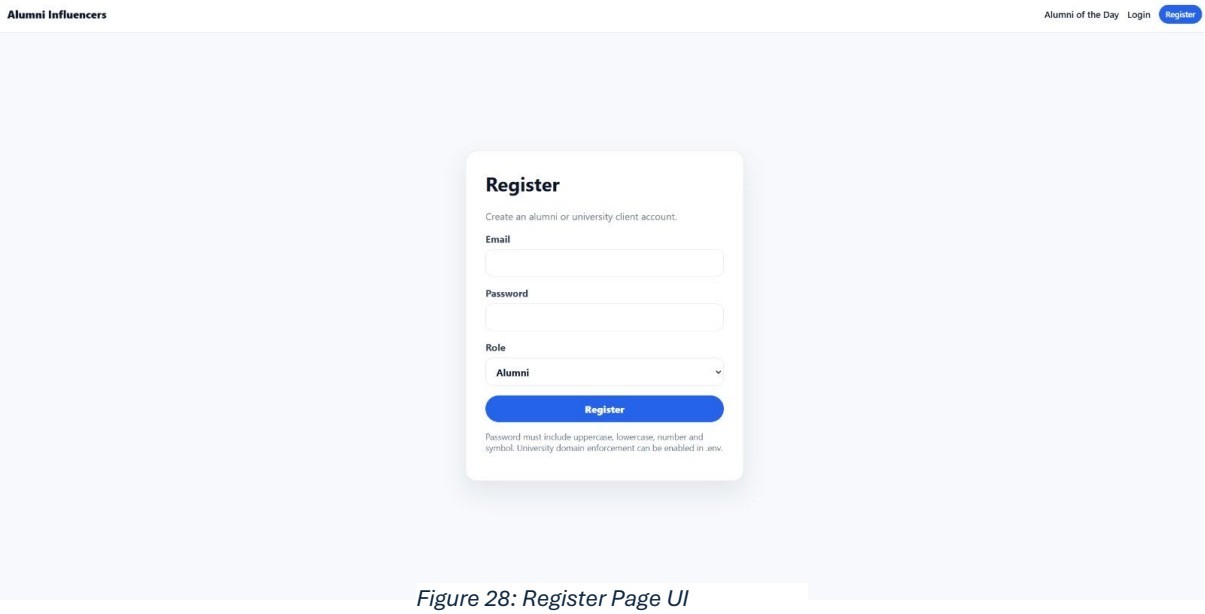

Figure 28: Register Page UI

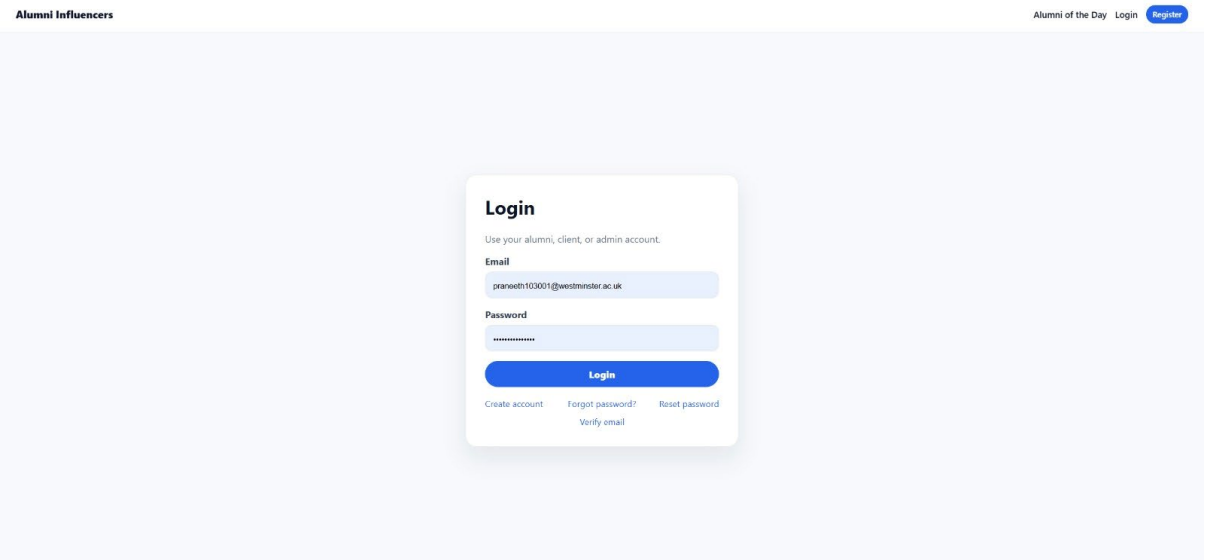

Figure 29: Login Page UI

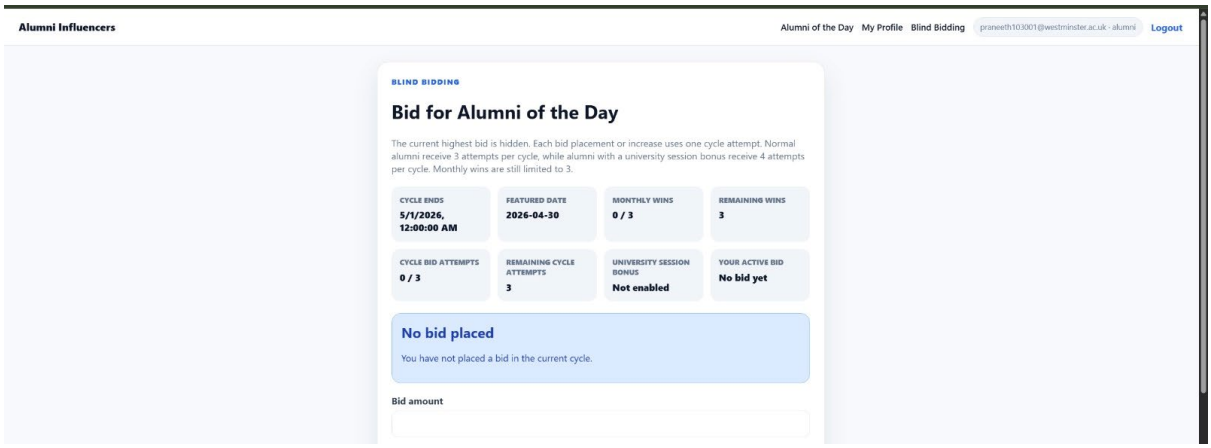

Figure 30: Bidding Page UI

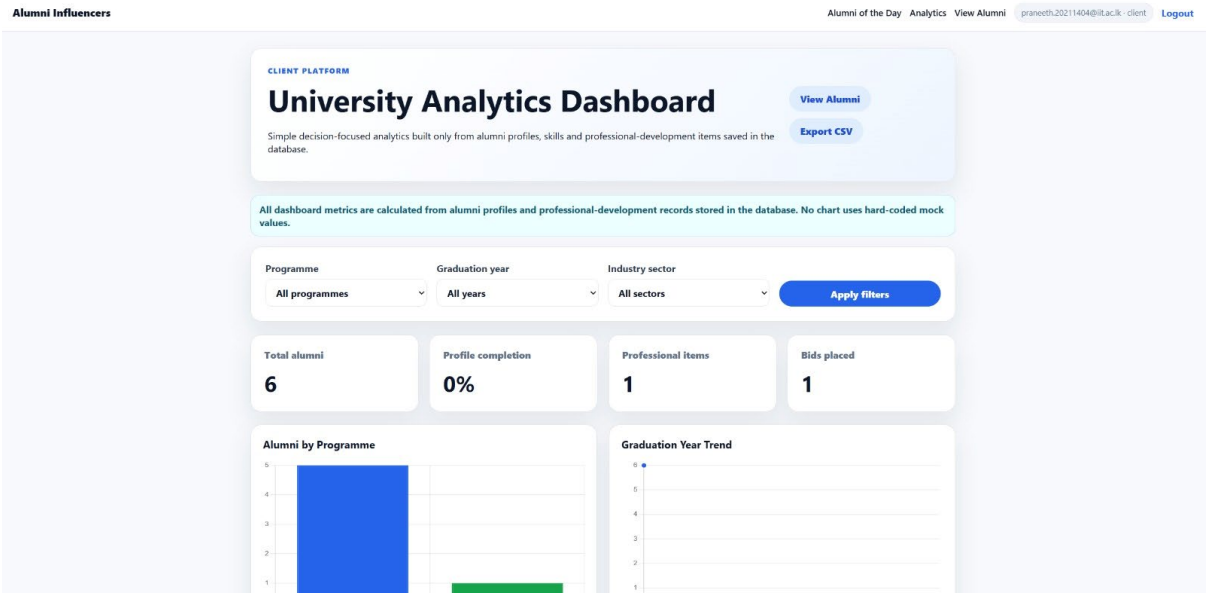

Figure 31: Data Analytics Dashboard UI

13. System Workflow

1. A new user registers using the public registration endpoint.
2. The system generates a verification token and confirms account ownership.
3. The verified user logs in and receives a bearer token.
4. The user creates or updates a professional alumni profile.
5. The user places a blind bid through the protected bidding endpoint.
6. The backend validates the bid against monthly and event-based rules.
7. The system stores the bid in the database and keeps it hidden from competitors.
8. When the cycle closes, winner selection logic runs and produces the final outcome.

14. Conclusion

This report presented the design and implementation of a dynamic alumni API platform developed to move beyond a static alumni system. The solution introduces secure authentication, structured profile management, blind bidding rules, public API exposure, and a scalable backend architecture built around clean service separation and database normalization.

From a software engineering perspective, the project demonstrates practical learning in REST API design, route organization, authentication, business rule enforcement, relational database design, and backend security implementation. It also provides a strong foundation for future expansion into real-time services, richer user interfaces, and more advanced deployment models.

15. Appendix

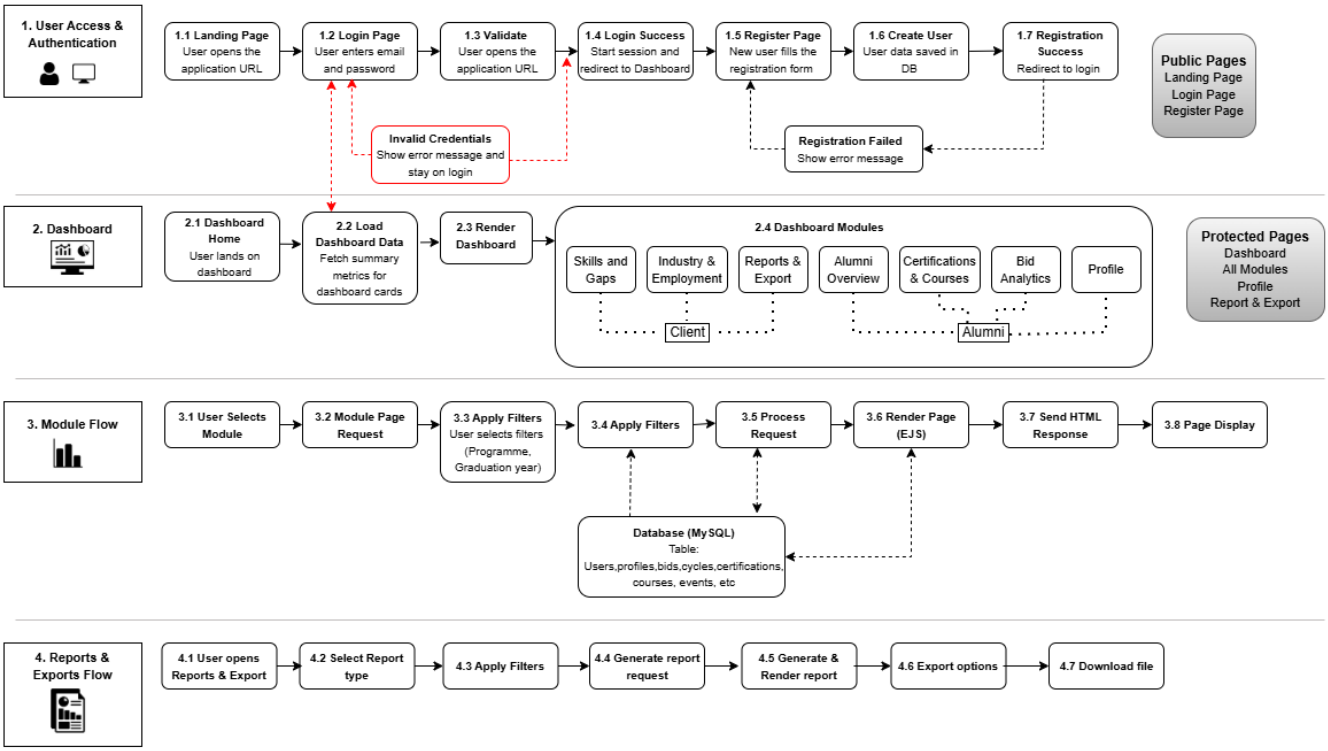

Figure 32: Descriptive System Workflow Diagram

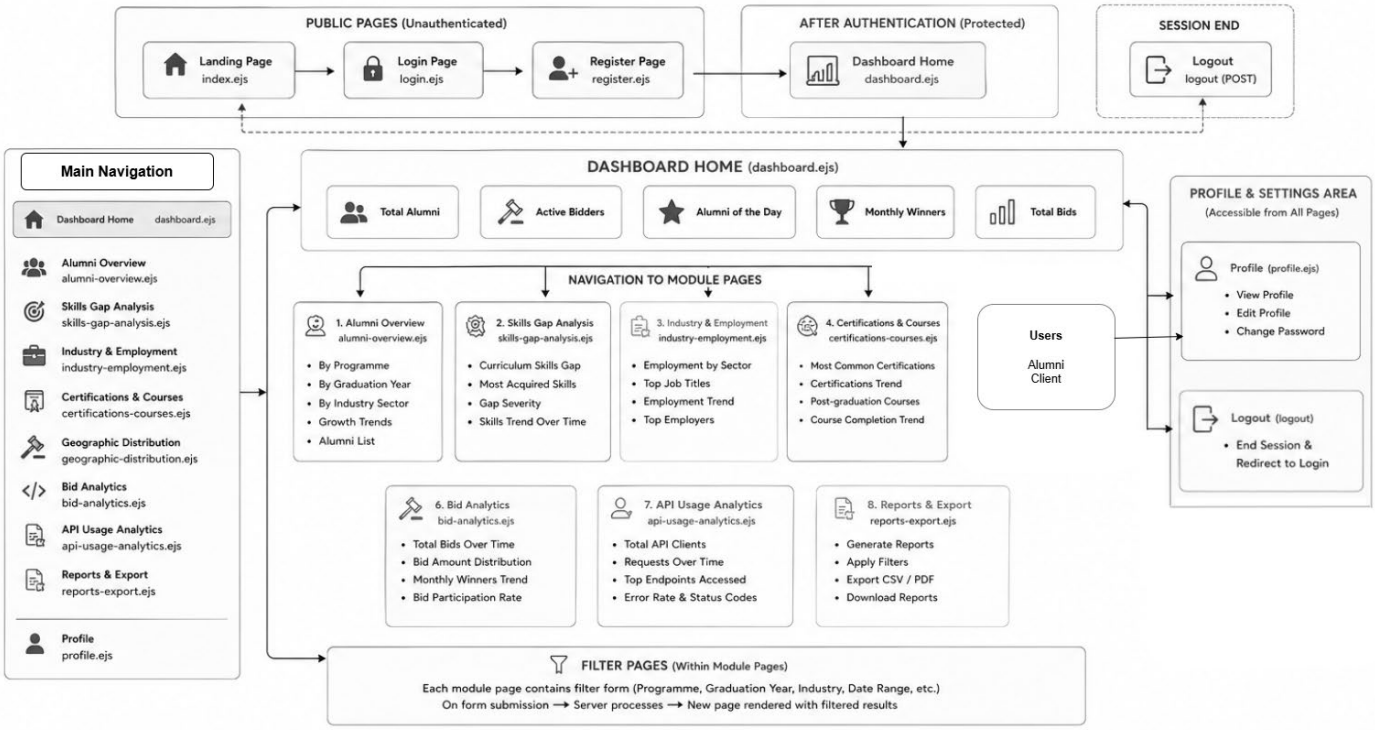

Figure 33: Descriptive Dashboard Workflow Diagram